Decentralized Authorized Artificial Intelligence Queries in Cloud-based Internet of Things

Mingyang Zhao, Junhan Qin, Xixi Zheng, Haojun Xuan, Wenbin Zhai, and Bin Xiao, *Fellow, IEEE*

***Abstract*—Artificial intelligence (AI) queries leverage AI models to generate predictive results and provide deeper insights than traditional match-based queries, which have shown significant potential for Internet of Things (IoT) applications. For example, in smart healthcare applications, users utilize IoT devices (e.g., watches or phones) to collect their health data, and doctors execute AI queries for disease prediction. Currently, to comply with the right to restriction of processing under laws and regulations, existing AI query works have to trust and depend on centralized cloud servers to enforce users' access control policies for authorized AI queries. However, this is infeasible for decentralized IoT environments, since these cloud servers, as enforcers, are not trusted and might bypass users' policies to break data security.**

In this paper, we introduce Decart, a decentralized authorized AI query scheme for cloud-based IoT applications. Different from existing works, while utilizing cloud servers for data outsourcing and AI queries, Decart enables users to formulate account-level policies that regulate which accounts can execute AI queries, and leverages cryptographic tools to enforce these policies on all entities, including these cloud servers. Meanwhile, Decart supports multiple AI models, such as decision trees and neural networks, and revocation functionalities to revoke semi-trusted users. Further, we optimize the size of users' stored parameters of Decart ($\mathcal{O}(n^2)$) to introduce a more efficient variant as Decart* ($\mathcal{O}(n)$), where n is related to the number of users. Security analysis demonstrates the security in decentralized settings. Experiments show that, on large-parameter AI models, compared with a naive composition of existing techniques for authorized AI queries, Decart and Decart* reduce online computation costs by 1.19× and 1.24×. The source code is outsourced at GitHub[1].

***Index Terms*—Internet of Things, Cloud Servers, Artificial Intelligence Queries, Authorization, and Decentralization.**

I. Introduction

Artificial Intelligence (AI) Query [1], [2] refers to the use of artificial intelligence models to reason about input data and output prediction results. Compared with the traditional query scheme based on exact matching, it has stronger capabilities. This advantage gives AI query great potential in the application of the Internet of Things (IoT) [3], [4]. For example, in the smart healthcare scenario, doctors can predict diseases based on health data collected by wearable devices [5]; in the industrial IoT scenario, operation and maintenance personnel can use equipment operation data for anomaly diagnosis [6]; in the smart home scenario, the system can predict potential risk events based on environmental and behavioral data [7]; in the vehicular IoT scenario, traffic management systems can predict traffic flow based on connected vehicle data [8]. With the continuous generation of large-scale data produced by IoT terminals, the emergence of AI queries has been greatly facilitated [9], [10].

Meanwhile, with the promulgation of various laws and regulations, such as GDPR [11] and PIPL [12], returning data usage rights to users themselves has become a key requirement in the development of AI query [13]. To meet this requirement, the current schemes [14], [15] are to allow users to formulate policies to control which accounts can perform AI queries. Then, these policies are sent to the cloud server, which is responsible for executing the policies and providing corresponding services. However, such a design relies heavily on the honesty of the cloud server. In a decentralized IoT environment, this assumption is often unrealistic [16], [17]. That's because in a decentralized IoT environment, the cloud server is not inherently trustworthy, it may ignore user policies and privately provide unauthorized query services, thereby do harm to users' data security and access control goals. Therefore, how to achieve mandatory authorization over AI query in an untrusted cloud environment has become a key issue that urgently needs to be addressed [18].

For this issue, current studies have made preliminary efforts. For example, Liang et al. [19] proposed a scheme called SecPQ, which uses symmetric encryption and key distribution to enforce mandatory query control, thereby avoiding complete reliance on the cloud server for policy enforcement. However, the use of symmetric encryption requires offline key distribution, which not only increases system complexity but also introduces the risk of key leakage. On the other hand, to support more powerful AI models, another direction is simply combining existing techniques [20], [21]. There are possible solutions: (1) homomorphic encryption (HE) with cloud-based access control policies and owner decryption [22], (2) homomorphic encryption with offline key distribution and owner decryption [23], and (3) CCS-style registration-based authorization with HE-based AI query execution [24], [25]. The first solution is vulnerable to smuggling of AI services, while the second suffers from key leakage and requires online interaction with the data owner during decryption. The third inherits the registration overhead and limited practical flexibility of a lightweight RBE-style construction when adapted to AI query settings. Furthermore, none of the above solutions support policy revocation. This shows that existing techniques are hard to simultaneously satisfy mandatory authorization,

M. Zhao, J. Qin, X. Zheng, W. Zhai, X. Cai, and B. Xiao are with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong. Email: {25019897r}@connect.polyu.hk and {b.xiao}@polyu.edu.hk.

Corresponding author: Bin Xiao (Email: b.xiao@polyu.edu.hk).

This work was supported in part by the National Natural Science Foundation of China (NSFC) / Research Grants Council (RGC) Collaborative Research Scheme (Grant No. 62461160332 & CRS_HKUST602/24).

[1]https://github.com/Academic-Paper-Codes/Decart

support for complex models, revocation capability, and practical deployability [26], [27].

To solve above issues, in this paper, we propose Decart, a decentralized authorized AI query scheme for cloud-based IoT applications. The core idea of Decart is to bind account registration, access policy constraints, and the query key recovery process together, so that only legitimate registered users who satisfy the policies can recover a valid query key and further perform AI queries. Decart allows users to define account-level access policies and enforces these policies on all entities by using cryptographic mechanisms, thereby avoiding the risk of a centralized server bypassing policies or abusing its privileges. Compared with existing works [19], [22], [23], Decart offers three advantages. First, Decart does not rely on offline key distribution, which reduces the risk of key leakage and improves deployment flexibility. Second, Decart supports more powerful AI models. Third, Decart supports user revocation, allowing users' AI query permissions to be withdrawn in a timely manner when their permissions change or after semi-trusted behavior occurs. Based on Decart, we further propose Decart*, which reduces overhead and improves scalability by optimizing user-side parameter storage and system execution processes. Based on the above design, the main contributions of this paper are as follows.

1. This paper proposes Decart, a decentralized authorized AI query scheme for IoT applications. Decart allows users to define account-level access policies and enforces mandatory authorization for AI queries in untrusted environments. It avoids reliance on offline key distribution and supports authorization revocation.

2. Based on Decart, this paper further designs a more efficient variant, Decart*, which further optimizes user-side storage and system overhead, thereby improving the scheme's scalability in large-scale user scenarios.

3. A systematic security analysis and experimental evaluation were conducted on the proposed schemes. The analysis confirms that Decart and Decart* satisfy the expected security objectives in decentralized environments. The experimental results further show that, when compared with naive baseline solutions built from existing techniques, Decart and Decart* strike an excellent balance between computational cost, communication cost, and functional support, thereby delivering competitive performance.

The remainder of this paper is organized as follows. Section II introduces related work. Section III provides relevant definitions. Section IV presents the system model and threat model. Section V describes the design details of Decart and Decart*, along with their correctness and complexity analysis. Section VI presents the security analysis. Section VII provides the experimental evaluation. Finally, Section VIII concludes the paper.

II. Related Works

Existing research on AI queries mainly focuses on how to support efficient model inference in the cloud and how to provide prediction services while protecting data privacy [1]–[3]. For this purpose, some studies combine cryptographic tools such as secure multi-party computation and homomorphic encryption to achieve privacy-preserving inference without exposing raw data [20], [21], [28]. These studies lay the foundation for the secure implementation of AI queries. However, most current works primarily focus on protecting data content and the inference process itself, while paying less attention to who is authorized to perform AI queries—that is, lacking systematic support for query authorization control. Especially in decentralized IoT environments, protecting data privacy without controlling query permissions still fails to meet users' requirements for data usage rights.

To meet the requirements of data regulations regarding the right to restrict data processing, existing studies have begun to focus on the issue of authorized AI queries, where data owners define access control policies to limit which accounts can perform AI queries. An scheme is to delegate access control policy enforcement to the cloud server: users send policies to the server, which then the server determines whether a querier has the appropriate permissions based on those policies. Such an scheme is simple to implement and compatible with existing cloud service architectures. However, its core limitation is that it heavily relies on the trustworthiness of the cloud server. In decentralized IoT scenarios, the server is not inherently trustworthy and may bypass policies to provide unauthorized query services, making truly mandatory access control difficult to achieve.

To address this issue, Liang et al. proposed SecPQ [19], which achieves controlled AI queries through symmetric encryption and key distribution mechanisms. Compared with schemes that fully rely on servers for policy enforcement, this work enhances the mandatory nature of authorization control to some extent. However, SecPQ still has several limitations. First, it relies on offline key distribution, which not only increases system deployment complexity but also introduces the risk of key leakage. Second, SecPQ only supports relatively simple AI models such as decision trees, making it difficult to meet the requirements of more complex models like neural networks in IoT scenarios. Third, the scheme does not support user revocation. Consequently, when a semi-trusted user has already obtained a valid key, the system cannot promptly revoke their query capability.

Besides schemes discussed above, several naive combinations can be built by directly stitching together existing cryptographic techniques [22], [23]. For instance, homomorphic encryption can be combined with online access control policies and owner-side decryption, or with offline key distribution to enable restricted query access. However, these naive schemes either rely on online policy enforcement—making it difficult to prevent untrusted servers from bypassing access control or smuggling AI services—or inherit the key leakage risks of offline key distribution. Furthermore, they generally lack user revocation support and fail to balance multi-model compatibility with system efficiency.

In summary, existing work has not simultaneously satisfied the following requirements: enforcing mandatory authorization control for AI queries in decentralized environments, avoiding offline key distribution, supporting rich AI models, and providing user revocation functionality. To fill this gap, this paper

TABLE I: Comparison between Representative AI Query Baselines and Our Proposed Solutions.

Works	Authorization	Rich AI Models	No Owner Involvement	No Offline Key Distribution	No Smuggling	Decentralization
Trusted-Server Plaintext AI Query		✓	✓	✓		
Symmetric-Encryption-Based SecPQ [19]	✓		✓		✓	
HE with Cloud-Based Control [22]	✓	✓		✓		
HE with Offline Key Distribution [23]	✓	✓				
CCS-Style Registration-Based AI Query [24], [25]	✓	✓	✓	✓	✓	
Our Solutions	✓	✓	✓	✓	✓	✓

proposes Decart and Decart.*

III. Preliminaries

A. Artificial Intelligence and Prediction Queries

An AI query is a query scheme that uses an artificial intelligence model to perform inference on an input and produce a prediction result, classification label, or decision outcome [20]. Formally, let the query input be denoted as q and the model as $f(\cdot)$. The prediction result can then be expressed as $y = f(q)$.

This paper aims to provide a solution to the problem of authorized AI queries in cloud-based IoT environments. In this setting, data owners outsource models and related data to the cloud, while data queriers can obtain valid prediction results only when they satisfy the access control policies. The schemes proposed in this paper are applicable to a variety of AI models, including dot product models, decision tree models, and neural network models.

B. Bilinear Maps and Hardness Assumptions

Let $\Psi = (p, G, G_T, g, e)$ be a bilinear group setting, where G and G_T are cyclic groups of prime order p, g is a generator of G, and $e : G \times G \to G_T$ is a bilinear map satisfying bilinearity ($e(g^a, g^b) = e(g, g)^{ab}$), non-degeneracy ($e(g, g) \neq 1_{G_T}$), and efficient computability [29], [30]. We further employ a hash function $H : G_T \to \{0, 1\}^\lambda$ to derive bit strings for query-key masking. Our security analysis relies on the **Square-BDH assumption**: given (g, g^a, g^b) for uniformly random $a, b \in \mathbb{Z}_p$, no probabilistic polynomial-time adversary can compute $e(g, g)^{a^2 b}$ with non-negligible probability.

C. Registration-based Encryption

Registration-based encryption (RBE) binds decryption capability to user registration [24], [25]. A typical RBE scheme consists of Setup, KeyGen, Register, Encrypt, and Decrypt [24]. The key idea is that decryption eligibility depends not only on holding a static private key but also on legitimate registration and satisfaction of access conditions [25]. In this paper, we adopt the idea of RBE to achieve account-level authorization for AI queries: data owners embed access policies into ciphertexts so that only queriers who satisfy the policies and have completed registration can recover valid query keys. If a user is revoked, the system updates relevant parameters to invalidate their registration, thereby revoking query permissions.

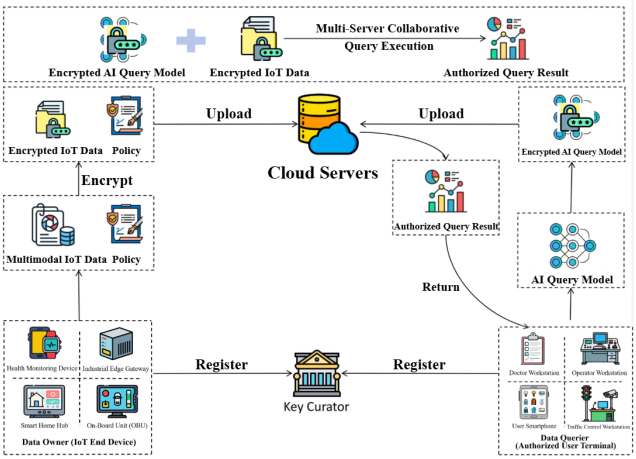


Fig. 1: The system model in this paper.

D. Threshold Homomorphic Encryption

Threshold homomorphic encryption (THE) combines homomorphic computation with threshold decryption [31], [32]. Homomorphism allows servers to perform computations (e.g., addition and multiplication) directly on ciphertexts without decryption [31], while threshold decryption distributes decryption capability among multiple parties, preventing any single entity from holding the full private key [32]. Only collaboration meeting the threshold requirement enables legitimate decryption [32]. In this paper, THE enables privacy-preserving AI query execution in the cloud: data owners encrypt their data, queriers encrypt their AI models after recovering a valid query key, and the cloud server performs homomorphic evaluation to produce encrypted results, which can only be fully decrypted by authorized queriers.

IV. Problem Formulation

In this section, we present the system model and threat model in this paper. Then, we summarize the design goals.

A. System Model

As shown in Fig. 1, the system model in Web 3.0 artificial intelligence query scenarios consists of the key curator, data owners, cloud servers, and data queriers.

- **Key Curator.** Without any secrets, the key curator is a public platform for collecting user keys and maintaining system parameters.
- **Data Owner.** Data owners, as Web 3.0 users, store data records to the cloud server and formulate query policies for managing AI queries.

- **cloud server.** As cloud servers, the cloud server is responsible for storing the data owners' data records and executing data queriers' query services.
- **Data Querier.** As Web 3.0 users, data queriers use the cloud server to execute AI queries.

B. Threat Model

The semi-trusted assumption is adopted in this paper, meaning that all entities in the system model are semi-trusted. For instance, the key curator is curious about all users' secret keys, the data owners' data records, and data queriers' AI models. Data owners are curious about other data owners' data records and data queriers' AI models. Data queriers attempt to break the query policy for unauthorized AI queries. The cloud server attempts to break the query policy to smuggle AI queries to unauthorized data queriers, and is curious about the data queriers' AI models. Collusion attacks might exist between these entities, and solutions against collusion attacks are discussed in Section VI.

C. Design Goals

Without leaking data owners' data records and data queriers' AI models, the cloud server combines data owners' query policies to execute AI queries and returns the query results to data queriers. Specifically, we summarize the design goals in terms of functionality, security, and efficiency.

- **Functionality.** Data owners can control which data queriers can execute AI queries.
- **Security.** Without any centralized entity, data owners' query policies are enforceable to all entities, including the cloud server. This means, only data queriers whose identities satisfy the data owners' query policies can execute AI queries, which prevents the cloud server from smuggling AI queries to unauthorized data queriers. When semi-trusted users are reported, the key curator can execute revocation functionalities to remove them.
- **Efficiency.** Operations with heavy computations, such as the AI query processes, occur on the cloud server, instead of data owners and data queriers. Meanwhile, towards multiple different data queriers, data owners simply need to generate a $|m|$-sized ciphertext, rather than generating a $|m|$-sized ciphertext for each data querier, where $|m|$ denotes the size of data records.

V. Our Proposed Decart and Decart*

Combining the above idea and definition of our schemes, we present the detailed constructions below.

A. Main Idea for Technical Challenges

Idea for Transferring the Focus of Registration-based Encryption from Data to AI Query Control. Traditional RBE is used to encrypt data for specific registered users. In this paper, we change its focus to AI query authorization. We combine account registration, policy rules, and query key recovery together. As a result, only users who have registered and satisfy the policy can recover a valid query key to run AI queries. This design achieves mandatory authorization without needing to trust the cloud server.

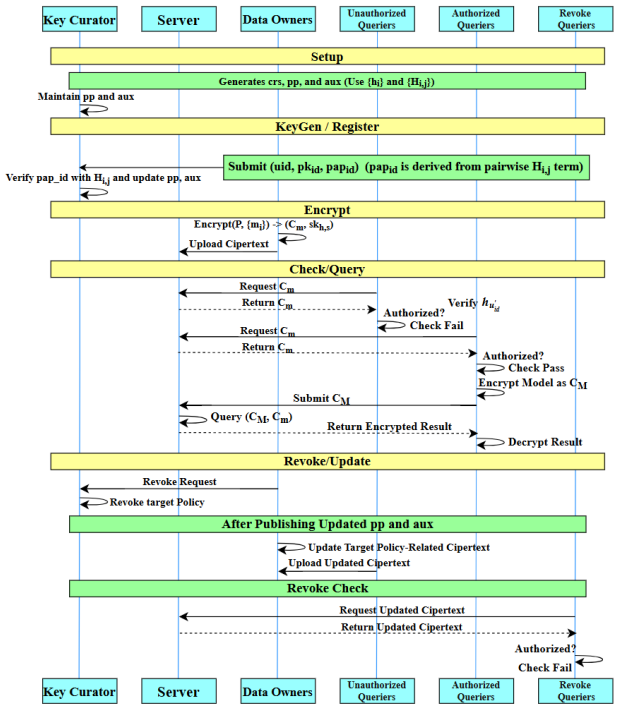


Fig. 2: Interaction Workflow of DeCart.

Idea for Reducing Communication Costs towards Multi-user Scenarios. In a multi-user setting, a simple method would require data owners to create a separate ciphertext for each authorized querier. This brings high communication cost. To solve this problem, our scheme packs policy-related information into a small number of ciphertext components. No matter how many authorized users there are, the data owner only needs to generate $O(n_p)$ policy parameters, where n_p is the policy size. This is much better than generating $O(N)$ ciphertexts for each querier.

Idea for Reducing the Size of Owners' Stored Parameters. In Decart, each data owner needs to store $O(n^2)$ auxiliary parameters because of how the parameters are organized. Decart* improves this by reorganizing the public and auxiliary parameters using an exponent-chain structure. This reduces the storage cost from $O(n^2)$ to $O(n)$. This improvement makes Decart* more scalable in IoT environments with many users.

B. Detailed Construction for Decart

As shown in Fig.2, the interaction workflow of DeCart consists of the Setup, KeyGen/Register, Encrypt, Check/Query, and Revoke/Update phases.

Setup $(\lambda) \rightarrow (crs, pp, aux)$: Given a security parameter λ, the system start the trusted initialization. First, the system generates a bilinear group Ψ as (p, g, G, G_T, Z_p, e), where $e(G, G) \rightarrow G_T$. Let the parameter $N \in Z_p$ as the maximum number of users, $n \in Z_p$ as the number of users in a block, and $B = \lceil \frac{N}{n} \rceil$ as the number of blocks. Under a trusted setup, the

system samples n random values $\{z_i \in Z_p\}_{i=1}^{n}$, and computes the following parameters.

$$h_i = g^{z_i},\ i = 1, 2, \cdots, n,$$
$$H_{i,j} = g^{z_i \cdot z_j},\ i, j = 1, 2, \cdots, n,\ and\ i \neq j.$$

Finally, the system selects a hash function $H[G_T] \rightarrow \{0,1\}^*$ and initializes the common reference string crs, the public parameter pp, and the auxiliary parameter aux as follows.

$$crs = (\Psi, N, B, n, \{h_i\}_{i\in[n]}, \{H_{i,j}\}_{i,j\in[n]\&i\neq j}, H[\cdot]),$$
$$pp = (C_{(1)} = 1, C_{(2)} = 1, \cdots, C_{(B)} = 1) \in G,$$
$$aux = (L_1 = \{1\}, L_2 = \{1\}, \cdots, L_N = \{1\}) \in G.$$

KeyGen $(u_{id}) \rightarrow (sk_{id}, pk_{id}, pap_{id})$: Given the user identity $u_{id} \in Z_p$, the user samples a random value $x_{id} \in Z_p^*$, and sets $u'_{id} = (u_{id} \mod n) + 1$. Then, the user generates his/her own secret key sk_{id}, public key pk_{id}, and personal auxiliary parameter pap_{id} as follows. Next, the user sends the tuple $(u_{id}, pk_{id}, pap_{id})$ to the key curator for registration.

$$sk_{id} = x_{id},\ pk_{id} = h_{u'_{id}}^{x_{id}},$$
$$pap_{id} = (H_{1,u'_{id}}^{x_{id}}, \cdots, H_{u'_{id}-1,u'_{id}}^{x_{id}}, \phi, H_{u'_{id}+1,u'_{id}}^{x_{id}}, \cdots, H_{n,u'_{id}}^{x_{id}}).$$

Register $(u_{id}, pk_{id}, pap_{id}) \rightarrow (pp', aux')$: Receiving tuple $(u_{id}, pk_{id}, pap_{id})$, the key curator checks the user identity u_{id} and personal auxiliary parameter pap_{id} as follows.

$$e(pk_{id}, h_1) \stackrel{?}{=} e(pap_{id,1}, g), \cdots, e(pk_{id}, h_n) \stackrel{?}{=} e(pap_{id,n}, g),$$

where $u'_{id} = (u_{id} \mod n)+1$. If the above checking process holds, the key curator computes the parameter $k = \lceil \frac{u_{id}+1}{n} \rceil$ as the block number belonging to u_{id}. Then, the key curator updates $C'_{(k)}$ as $C_{(k)} \cdot pk_{id}$. Apart from $C_{(k)}$, other parameters in public parameter pp remain the same, i.e., the updated pp' is $(C_{(1)}, C_{(2)}, \cdots, C'_{(k)}, \cdots, C_{(B)})$. Next, for $j \in \{(k-1)n+1, (k-1)n+2, \cdots, (k-1)n+n\} \backslash (u_{id}+1)$, the key curator parses L_j as $(O_{j,1}, O_{j,2}, \cdots, O_{j,o})$ and updates L_j as $L_j \cup \{O_{j,o+1} = O_{j,o} \cdot pap_i\}$, where o denote the current size of L_j and $i = j - (k-1)n$. Apart from $j \in \{(k-1)n+1, (k-1)n+2, \cdots, (k-1)n+n\} \backslash (u_{id}+1)$, other parameters in the auxiliary parameter aux remains the same.

Encrypt $(\mathcal{P}, \{m_i\}_{i=1}^{n_m}) \rightarrow C_m$: The user formulates the access policy $\mathcal{P}$ as a series of user identities $\{u_{id,i}\}_{i=1}^{n_p}$, where n_p denotes the size of the access policy. Then, the user samples a random value $\alpha \in Z_p$ and a homomorphic encryption key tuple (pk_h, sk_h) with the homomorphic encryption as $Enc[\cdot]$, homomorphic decryption as $Dec[\cdot]$, and homomorphic evaluation as $Eval[\cdot]$. Next, the user samples two random value $(\beta, \gamma) \in Z_p$, splits the homomorphic secret key sk_h as two shares $(sk_{h,s}, sk_{h,u})$, and encrypt the data m_i in the dataset $\{m_i \in \{0,1\}^*\}_{i=1}^{n_m}$ as follows, where n_m denotes the number of data records.

$$k_i = \lceil \frac{u_{id,i}+1}{n} \rceil,\ i = 1, 2, \cdots, n_p,$$
$$c_{1,i} = C_{(k_i)},\ i = 1, 2, \cdots, n_p,$$
$$u'_{id,i} = (u_{id,i} \mod n) + 1,\ i = 1, 2, \cdots, n_p,$$
$$c_{2,i} = e(C_{(k_i)}, h_{u'_{id}})^{\gamma},\ i = 1, 2, \cdots, n_p,$$
$$c_3 = g^{\gamma},$$
$$c_{4,i} = H[e(h_{u'_{id}}, h_{u'_{id}})^{\gamma}] \oplus \beta,\ i = 1, 2, \cdots, n_p,$$
$$c_5 = H[e(g,g)^{\beta}] \oplus (pk_h || sk_{h,u}),$$
$$c_{6,i} = Enc[pk_h, m_i],\ i = 1, 2, \cdots, n_m.$$

Finally, the user packs the ciphertext C_m as $(\mathcal{P}, \{c_{1,i}\}_{i=1}^{n_p}, \{c_{2,i}\}_{i=1}^{n_p}, c_3, \{c_{4,i}\}_{i=1}^{n_p}, c_5, \{c_{6,i}\}_{i=1}^{n_m})$, and sends the tuple $(C_m, sk_{h,s})$ to the cloud server. Then, the cloud server retains the policy-related parameters $(\mathcal{P}, \{c_{1,i}\}_{i=1}^{n_p}, \{c_{2,i}\}_{i=1}^{n_p}, c_3, \{c_{4,i}\}_{i=1}^{n_p}, c_5$ locally and uploads the data-related parameters $\{c_{6,i}\}_{i=1}^{n_m})$ to blockchains for storage. To prevent collusion attacks among users and the single cloud server, the user can introduce more cloud servers, split the key sk_h into more shares, and distribute a share to each cloud server. For example, the key is divided into $(n_c + 1)$ shares $(sk_{h,s,1}, sk_{h,s,2}, \cdots, sk_{h,s,n_s}, sk_{h,u})$, each share in $(sk_{h,s,1}, sk_{h,s,2}, \cdots, sk_{h,s,n_s})$ is transmitted to a cloud server. Then, during decryption, these cloud servers or above-threshold cloud servers need to collaborate to execute partial decryption.

Check $(u_{id}, sk_{id}, C_m) \rightarrow C_M$: Based on the user identity u_{id} and the ciphertext C, the user checks whether u_{id} satisfies the access policy in C as $u_{id} \stackrel{?}{\in} \mathcal{P}$. Then, the user parses the parameter L_{id} as $(O_{id,1}, O_{id,2}, \cdots)$, and obtains the parameters in C belonging to him/her (e.g., the index is $j \in [n_p]$), where o denotes the size of L_{id}. Then, the user checks whether an index i exists that satisfies the following equation.

$$e(c_{1,j}, h_{u'_{id}}) \stackrel{?}{=} e(O_{id,i}, g) \cdot e(h_{u'_{id}}^{sk_{id}}, h_{u'_{id}}).$$

If there is no such i, the user executes the **Update** phase for the up-to-date parameter L_{id}. Otherwise, for the index i, the user decrypts the homomorphic secret key pk_h as follows.

$$\beta = c_{4,j} \oplus H[(e(O_{id,i}, c_3)^{-1} \cdot c_{2,j})^{\frac{1}{sk_{id}}}],$$
$$pk_h || sk_{h,u} = c_5 \oplus H[e(g,g)^{\beta}].$$

Next, the user recovers the keys $(pk_h, sk_{h,u})$, encrypts his/her AI mode M as C_M, and sends C_M to the cloud server for AI queries. As shown in Algorithm 1 and Algorithm 2, we use decision trees and feed-forward neural networks as two examples to illustrate the model encryption process.

$$C_M = Enc[pk_h, M].$$

Query $(C_M, C_m) \rightarrow ER$: With the encrypted AI model C_M, the cloud server queries the data owners' encrypted dataset $\{c_{7,i}\}_{i=1}^{n_m}$. Specifically, the cloud server executes $Eval[C_M, \{c_{7,i}\}_{i=1}^{n_m}]$ and leverages the parameter $sk_{h,s}$ to gain the encrypted query result ER as $Dec[sk_{h,s}, Eval[C_M, \{c_{7,i}\}_{i=1}^{n_m}]]$. Next, the result ER is returned to the user. Here, as mentioned in the **Encrypt** algorithm, if the data owner chooses to split the key sk_h to

Algorithm 1: Threshold Homomorphic Encryption-based Model Encryption Algorithm (Take Decision Trees as Examples).

Require: Decision tree model $\mathcal{T}$ with internal nodes $\mathcal{N}$ and leaf nodes $\mathcal{L}$. Each internal node $u \in \mathcal{N}$ is associated with a feature index j_u and a threshold θ_u; each leaf node $\ell \in \mathcal{L}$ is associated with an output value v_ℓ.

Ensure: Encrypted decision tree model $Enc(\mathcal{T})$.

A decision tree model $\mathcal{T}$ defines a mapping $x \mapsto \mathcal{T}(x)$ by recursively routing the input x from the root to a leaf, where $\ell^\star$is the reached leaf when following tests $x_{j_u} \overset{?}{\leq} \theta_u$ at each node $u \in \mathcal{N}$:

$$\mathcal{T}(x) = v_{\ell^\star}.$$

for each internal node $u \in \mathcal{N}$ **do**
  Encrypt the feature index:

$$\widetilde{j}_u \leftarrow Enc_{pk}(j_u).$$

  Encrypt the threshold:

$$\widetilde{\theta}_u \leftarrow Enc_{pk}(\theta_u).$$

end for
for each leaf node $\ell \in \mathcal{L}$ **do**
  Encrypt the leaf output value:

$$\widetilde{v}_\ell \leftarrow Enc_{pk}(v_\ell).$$

end for
Denote encrypted decision tree parameters as:

$$Enc(\mathcal{T}) = \left\{\widetilde{\theta}_u, \widetilde{j}_u\right\}_{u\in\mathcal{N}} \cup \left\{\widetilde{v}_\ell\right\}_{\ell\in\mathcal{L}}.$$

return $Enc(\mathcal{T})$.

Algorithm 2: Threshold Homomorphic Encryption-based Model Encryption Algorithm (Take Feed-Forward Neural Networks as Examples).

Require: Neural network model $\mathcal{M}$ with L layers and parameters $\{W_\ell, b_\ell\}_{\ell=1}^{L}$.

Ensure: Encrypted model $Enc(\mathcal{M})$.

The model $\mathcal{M}$ is a feed-forward neural network with L layers:

$$\mathcal{M}(x) = f_L\big(W_L \cdot f_{L-1}(\dots f_1(W_1 x + b_1)\dots) + b_L\big),$$

where f_ℓ are activation functions (e.g., polynomial or piecewise-linear functions that are HE-friendly).
for each layer $\ell = 1$ to L **do**
  for each entry $W_\ell(i,j)$ of weight matrix W_ℓ **do**
    Compute encrypted weight:

$$\widetilde{W}_\ell(i,j) \leftarrow Enc_{pk}\big(W_\ell(i,j)\big).$$

  end for
  for each entry $b_\ell(k)$ of bias vector b_ℓ **do**
    Compute encrypted bias:

$$\widetilde{b}_\ell(k) \leftarrow Enc_{pk}\big(b_\ell(k)\big).$$

  end for
end for
Denote encrypted model parameters as:

$$Enc(\mathcal{M}) = \left\{\widetilde{W}_\ell, \widetilde{b}_\ell\right\}_{\ell=1}^{L}.$$

return $Enc(\mathcal{M})$.

multiple cloud servers, above-threshold cloud servers need to collaborate to execute partial decryption over ER. Finally, the partially decrypted ER is returned to the user. Receiving the encrypted query result ER, the user utilizes the parameter $sk_{h,u}$ to decrypt ER by $Dec[sk_{h,c}, ER]$ and gains the query result R. As shown in Algorithm 3 and Algorithm 4, we use decision trees and feed-forward neural networks as two examples to illustrate the AI query process.

Update $(u_{id}) \rightarrow L_{id}$: Given the user identity u_{id}, the user gains the up-to-date public parameter L_{id} in the public auxiliary parameter aux.

Revoke $(u_{id}, pp, aux) \rightarrow (pp', aux')$: Given the revoked user identity u_{id}, the key curator is responsible for executing user revocation. Specifically, the key curator first generates a revocation factor as $r_{id} \in Z_p^*$, and then runs the **KeyGen** algorithm to gain the following parameters $(pk_{r,id}, pap_{r,id})$.

$$pk_{r,id} = h_{u'_{id}}^{r_{id}},$$
$$pap_{r,id} = (H_{1,u'_{id}}^{r_{id}}, \cdots, H_{u'_{id}-1,u'_{id}}^{r_{id}}, \phi, H_{u'_{id}+1,u'_{id}}^{r_{id}}, \cdots, H_{n,u'_{id}}^{r_{id}}).$$

Then, towards the user u_{id}, the key curator uses parameters $(pk_{r,id}, pap_{r,id})$ to run the **Register** algorithm and updates the parameters (pp', aux'). Meanwhile, data owners can choose to update the policy-related parameters $(\mathcal{P}', \{c'_{1,i}\}_{i=1}^{n_p}, \{c'_{2,i}\}_{i=1}^{n_p}, c'_3, \{c'_{4,i}\}_{i=1}^{n_p}, c'_5$, stored by the cloud server. After that, ciphertexts on up-to-date parameters $(pp', aux', \mathcal{P}', \{c'_{1,i}\}_{i=1}^{n_p}, \{c'_{2,i}\}_{i=1}^{n_p}, c'_3, \{c'_{4,i}\}_{i=1}^{n_p}, c'_5$ cannot be decrypted by the revoked user, meaning the user' query right is revoked.

C. Detailed Construction for Decart*

As shown in Fig.3, the interaction workflow of DeCart consists of the Setup, KeyGen/Register, Encrypt, Check/Query, and Revoke/Update phases.

Setup $(\lambda) \rightarrow (crs, pp, aux)$: Given a security parameter λ, the system generates a bilinear group Ψ as (p, g, G, G_T, Z_p, e), where $e(G, G) \rightarrow G_T$. Let the parameter $N \in Z_p$ as the maximum number of users, $n \in Z_p$ as the number of users in a block, and $B = \lceil \frac{N}{n} \rceil$ as the number of blocks. Under a trusted setup, the system samples a random value $z \in Z_p$, and computes the following parameters.

$$h_i = g^{z^i},\ i = 1, 2, \cdots, n, n+2, \cdots, 2n.$$

Finally, the system selects a hash function $H[G_T] \rightarrow \{0,1\}^*$ and initializes the common reference string crs, the public parameter pp, and the auxiliary parameter aux as follows.

$$crs = (\Psi, N, B, n, \{h_i\}_{i\in[2n]\ \setminus\ n+1}, H[\cdot]),$$
$$pp = (C_{(1)} = 1, C_{(2)} = 1, \cdots, C_{(B)} = 1) \in G,$$
$$aux = (L_1 = \{1\}, L_2 = \{1\}, \cdots, L_N = \{1\}) \in G.$$

Algorithm 3: Encrypted AI Query Algorithm (Take Decision Trees as Examples).

Require: Encrypted data records $\{c_{6,i}\}_{i=1}^{n_m}$, encrypted decision tree model $Enc(\mathcal{T}) = \{\widetilde{\theta}_u, \widetilde{j}_u\}_{u\in\mathcal{N}} \cup \{\widetilde{v}_\ell\}_{\ell\in\mathcal{L}}$, server's key share sk_s, querier's secret share sk_u.

Ensure: Querier obtains the plaintext query results R.

Server-side encrypted query:

for each selected encrypted input $c_{6,i}$ **do**

Let the encrypted feature vector for record i be:

$$\widetilde{\mathbf{x}}^{(i)} \leftarrow c_{6,i}.$$

Encrypted tree evaluation:

For each internal node $u \in \mathcal{N}$, the server computes the encrypted comparison result:

$$\widetilde{s}_u^{(i)} \leftarrow HE.Eval\big(g(\cdot), \widetilde{x}_{j_u}^{(i)}, \widetilde{\theta}_u\big),$$

where g is an HE-friendly function approximating the indicator of the predicate $[x_{j_u} \le \theta_u]$ (e.g., a low-degree polynomial).

Using $\{\widetilde{s}_u^{(i)}\}_{u\in\mathcal{N}}$, the server aggregates leaf-node contributions to obtain the encrypted output:

$$\widetilde{y}^{(i)} \leftarrow HE.Eval\Big(\sum_{\ell\in\mathcal{L}} \alpha_\ell(\{\widetilde{s}_u^{(i)}\}_{u\in\mathcal{N}}) \cdot \widetilde{v}_\ell\Big),$$

where $\alpha_\ell(\cdot)$ is an HE-friendly function that encodes the (soft) routing probability or indicator of reaching leaf ℓ based on the comparison results at internal nodes.

end for

The server returns the encrypted outputs:

$$ER \leftarrow HE.Dec\big[sk_s, \{\widetilde{y}^{(i)}\}\big].$$

for each ciphertext $\widetilde{y}^{(i)} \in ER$ **do**

The querier runs the threshold decryption protocol:

$$y^{(i)} \leftarrow HE.Dec\big(\widetilde{y}^{(i)}, sk_u\big).$$

end for

return $R = \{y^{(i)}\}$.

Algorithm 4: Encrypted AI Query Algorithm (Take Feed-Forward Neural Networks as Examples).

Require: Encrypted data records $\{c_{6,i}\}_{i=1}^{n_m}$, encrypted model $Enc(\mathcal{M}) = \{\widetilde{W}_\ell, \widetilde{b}_\ell\}_{\ell=1}^{L}$, server's key share sk_s, querier's key share sk_u.

Ensure: Querier obtains the plaintext query results R.

Server-side encrypted query:

for each selected encrypted input $c_{6,i}$ **do**

Initialize encrypted activation:

$$\mathbf{h}_0^{(i)} \leftarrow c_{6,i}.$$

for each layer $\ell = 1$ to L **do**

Encrypted linear transformation:

For each neuron k in layer ℓ, compute

$$\widetilde{z}_{\ell,k}^{(i)} \leftarrow HE.Eval\big(\widetilde{W}_\ell(k,\cdot), \mathbf{h}_{\ell-1}^{(i)}, \widetilde{b}_\ell(k)\big).$$

Collect ciphertext vector of pre-activations:

$$\widetilde{\mathbf{z}}_\ell^{(i)} \leftarrow \big(\widetilde{z}_{\ell,1}^{(i)}, \ldots, \widetilde{z}_{\ell,m_\ell}^{(i)}\big).$$

Encrypted activation function:

Apply an HE-friendly activation function f_ℓ:

$$\mathbf{h}_\ell^{(i)} \leftarrow HE.Eval\big(f_\ell, \widetilde{\mathbf{z}}_\ell^{(i)}\big).$$

end for

At the final layer L, obtain encrypted output:

$$\widetilde{y}^{(i)} \leftarrow \mathbf{h}_L^{(i)}.$$

end for

The server returns the encrypted outputs:

$$ER \leftarrow HE.Dec[sk_s, \{\widetilde{y}^{(i)}\}].$$

for each ciphertext $\widetilde{y}^{(i)} \in ER$ **do**

The querier runs the threshold decryption protocol:

$$y^{(i)} \leftarrow HE.Dec\big(\widetilde{y}^{(i)}, sk_u\big).$$

end for

return $R = \{y^{(i)}\}$.

KeyGen $(u_{id}) \to (sk_{id}, pk_{id}, pap_{id})$: Given the user identity $u_{id} \in Z_p$, the user samples a random value $x_{id} \in Z_p^*$, and sets $u'_{id} = (u_{id} \mod n) + 1$. Then, the user generates his/her own secret key sk_{id}, public key pk_{id}, and personal auxiliary parameter pap_{id} as follows. Next, the user sends the tuple $(u_{id}, pk_{id}, pap_{id})$ to the key curator for registration.

$$sk_{id} = x_{id},\ pk_{id} = h_{u'_{id}}^{x_{id}},$$
$$pap_{id} = (h_{u'_{id}+n}^{x_{id}}, h_{u'_{id}+n-1}^{x_{id}}, \cdots, h_{2+n}^{x_{id}}, \phi, h_n^{x_{id}}, \cdots, h_{u'_{id}+1}^{x_{id}}).$$

Register $(u_{id}, pk_{id}, pap_{id}) \to (pp', aux')$: Receiving tuple $(u_{id}, pk_{id}, pap_{id})$, the key curator checks the user identity u_{id} and personal auxiliary parameter pap_{id} as follows.

$$u_{id} \stackrel{?}{\in} [N],$$
$$e(pk_{id}, h_n) \stackrel{?}{=} e(pap_{id,1}, g) \stackrel{?}{=} \cdots \stackrel{?}{=} e(pap_{id,u'_{id}-1}, h_{u'_{id}-2})$$
$$\stackrel{?}{=} e(pap_{id,u'_{id}+1}, h_{u'_{id}}) \stackrel{?}{=} \cdots \stackrel{?}{=} e(pap_{id,n}, h_{n-1}),$$

where $u'_{id} = (u_{id} \mod n)+1$. If the above checking process holds, the key curator updates the public parameter pp and the auxiliary parameter aux like Decart.**Register** algorithm.

Encrypt $(\mathcal{P}, \{m_i\}_{i=1}^{n_m}) \to C_m$: The user formulates the access policy $\mathcal{P}$ as a series of user identities $\{u_{id,i}\}_{i=1}^{n_p}$, where n_p denotes the size of the access policy. Then, the user samples a random value $\alpha \in Z_p$ and a homomorphic encryption

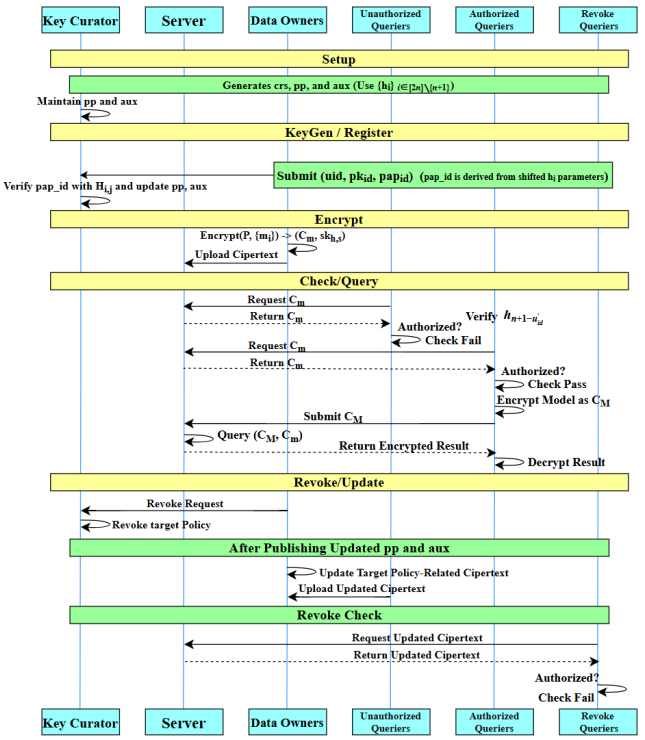


Fig. 3: Interaction Workflow of DeCart*.

key tuple (pk_h, sk_h) with the homomorphic encryption as $Enc[\cdot]$, homomorphic decryption as $Dec[\cdot]$, and homomorphic evaluation as $Eval[\cdot]$. Then, the user samples two random value $(\beta, \gamma) \in Z_p$, splits the homomorphic secret key sk_h as two shares $(sk_{h,s}, sk_{h,u})$, and encrypt the data m_i in the dataset $\{m_i \in \{0,1\}^*\}_{i=1}^{n_m}$ as follows, where n_m denotes the number of data records.

$$
\begin{aligned}
k_i &= \lceil \frac{u_{id,i}+1}{n} \rceil, \ i = 1, 2, \cdots, n_p, \\
c_{1,i} &= C_{(k_i)}, \ i = 1, 2, \cdots, n_p, \\
u'_{id,i} &= (u_{id,i} \mod n) + 1, \ i = 1, 2, \cdots, n_p, \\
c_{2,i} &= e(C_{(k_i)}, h_{n+1-u'_{id}})^{\gamma}, \ i = 1, 2, \cdots, n_p, \\
c_3 &= g^{\gamma}, \\
c_{4,i} &= H[e(h_{u'_{id}}, h_{n+1-u'_{id}})^{\gamma}] \oplus \beta, \ i = 1, 2, \cdots, n_p, \\
c_5 &= H[e(g,g)^{\beta}] \oplus (pk_h || sk_{h,u}), \\
c_{6,i} &= Enc[pk_h, m_i], \ i = 1, 2, \cdots, n_m.
\end{aligned}
$$

Finally, the user packs the ciphertext C_m as $(\mathcal{P}, \{c_{1,i}\}_{i=1}^{n_p}, \{c_{2,i}\}_{i=1}^{n_p}, c_3, \{c_{4,i}\}_{i=1}^{n_p}, c_5, \{c_{6,i}\}_{i=1}^{n_m})$, and sends the tuple $(C_m, sk_{h,s})$ to the cloud server. Then, the cloud server retains the policy-related parameters $(\mathcal{P}, \{c_{1,i}\}_{i=1}^{n_p}, \{c_{2,i}\}_{i=1}^{n_p}, c_3, \{c_{4,i}\}_{i=1}^{n_p}, c_5$ locally and uploads the data-related parameters $\{c_{6,i}\}_{i=1}^{n_m})$ to blockchains for storage. Similar to the **Encrypt** algorithm in Decart, to prevent collusion attacks among users and the cloud server, the user can introduce more cloud servers, split the key sk_h into $(n_s + 1)$ shares $(sk_{h,u}, sk_{h,s,1}, sk_{h,s,2}, \cdots, sk_{h,s,n_s})$, and distribute a share $sk_{h,s,i}$ to each cloud server s_i.

Check $(u_{id}, sk_{id}, C_m) \to C_M$: Based on the user identity u_{id} and the ciphertext C, the user checks whether u_{id} satisfies the access policy in C as $u_{id} \stackrel{?}{\in} \mathcal{P}$. Then, the user parses the parameter L_{id} as $(O_{id,1}, O_{id,2}, \cdots)$, and obtains the parameters in C belonging to him/her (e.g., the index is $j \in [n_p]$), where o denotes the size of L_{id}. Then, the user checks whether an index i exists that satisfies the following equation.

$$
e(c_{1,j}, h_{n+1-u'_{id}}) \stackrel{?}{=} e(O_{id,i}, g) \cdot e(h_{u'_{id}}^{sk_{id}}, h_{n+1-u'_{id}}).
$$

If there is no such i, the user executes the **Update** phase for the up-to-date parameter L_{id}. Otherwise, for the index i, the user decrypts the homomorphic secret key pk_h as follows.

$$
\begin{aligned}
\beta &= c_{4,j} \oplus H[(e(O_{id,i}, c_3)^{-1} \cdot c_{2,j})^{\frac{1}{sk_{id}}}], \\
pk_h || sk_{h,u} &= c_5 \oplus H[e(g,g)^{\beta}].
\end{aligned}
$$

Next, the user recovers the keys $(pk_h, sk_{h,u})$, encrypts his/her AI mode M as C_M, and sends C_M to the cloud server for AI queries. The model encryption process is similar to the **Check** algorithm in Decart, as shown in Algorithm 1 and Algorithm 2.

$$
C_M = Enc[pk_h, M].
$$

Update, **Query**, and **Revoke**: In Decart*, these algorithms are similar to those of Decart.

D. Correctness Analysis

We next show that DeCart and DeCart* are functionally correct. Specifically, for any honestly generated registration tuple and any authorized data querier whose identity satisfies the access policy, the querier can recover the correct query key material and finally obtain the correct AI query result.

a) Correctness of Register.: For a user with identity u_{id}, let

$$
u'_{id} = (u_{id} \bmod n) + 1.
$$

In the **KeyGen** phase, the user generates the secret key and public key as

$$
sk_{id} = x_{id}, \qquad pk_{id} = h_{u'_{id}}^{x_{id}},
$$

and generates the personal auxiliary parameters as

$$
pap_{id,j} = H_{j,u'_{id}}^{x_{id}}, \qquad j \neq u'_{id}.
$$

By the system setup, we have

$$
h_j = g^{z_j}, \qquad H_{j,u'_{id}} = g^{z_j z_{u'_{id}}}.
$$

Therefore, for any $j \neq u'_{id}$, we obtain

$$
e(pk_{id}, h_j) = e(h_{u'_{id}}^{x_{id}}, h_j) = e(g^{z_{u'_{id}} x_{id}}, g^{z_j}) = e(g,g)^{z_j z_{u'_{id}} x_{id}},
$$

and on the other hand,

$$
e(pap_{id,j}, g) = e(H_{j,u'_{id}}^{x_{id}}, g) = e(g^{z_j z_{u'_{id}} x_{id}}, g) = e(g,g)^{z_j z_{u'_{id}} x_{id}}.
$$

Hence,

$$
e(pk_{id}, h_j) = e(pap_{id,j}, g), \qquad \forall j \neq u'_{id}.
$$

This is exactly the verification equation required in the **Register** algorithm. Therefore, any registration tuple honestly generated according to the protocol can pass the **Register** verification, which shows the correctness of the registration procedure in DeCart.

b) Correctness of Authorized Query-Key Recovery.: Assume that a data querier identity u_{id} satisfies the access policy

$$P = \{u_{id,1}, u_{id,2}, \ldots, u_{id,n_p}\}.$$

Since u_{id} satisfies the access policy, there exists a matched policy index $j^\star$ such that

$$u_{id,j^\star} = u_{id}.$$

Then the corresponding intra-block index satisfies

$$u'_{id,j^\star} = (u_{id,j^\star} \bmod n) + 1 = (u_{id} \bmod n) + 1 = u'_{id}.$$

Therefore, the ciphertext components corresponding to this matched policy entry can be written as

$$c_{2,j^\star} = e(c_{1,j^\star}, h_{u'_{id}})^\gamma,$$

and

$$c_{4,j^\star} = H\Big(e(h_{u'_{id}}, h_{u'_{id}})^\gamma\Big) \oplus \beta.$$

On the other hand, since this querier is authorized, there exists a matched auxiliary entry $O_{id,i}$ in the auxiliary parameter set L_{id} such that the checking equation in the **Check** algorithm holds:

$$e(c_{1,j^\star}, h_{u'_{id}}) = e(O_{id,i}, g) \cdot e(h_{u'_{id}}^{sk_{id}}, h_{u'_{id}}).$$

Combining this equation with

$$c_{2,j^\star} = e(c_{1,j^\star}, h_{u'_{id}})^\gamma, \qquad c_3 = g^\gamma,$$

we obtain

$$c_{2,j^\star} = \Big(e(O_{id,i}, g) \cdot e(h_{u'_{id}}^{sk_{id}}, h_{u'_{id}})\Big)^\gamma.$$

By bilinearity, the above equation can be further expanded as

$$\begin{aligned} c_{2,j^\star} &= e(O_{id,i}, g^\gamma) \cdot e(h_{u'_{id}}^{sk_{id}}, h_{u'_{id}})^\gamma \\ &= e(O_{id,i}, c_3) \cdot e(h_{u'_{id}}, h_{u'_{id}})^{sk_{id}\gamma}. \end{aligned}$$

Hence,

$$e(O_{id,i}, c_3)^{-1} \cdot c_{2,j^\star} = e(h_{u'_{id}}, h_{u'_{id}})^{sk_{id}\gamma}.$$

Taking both sides to the power of $1/sk_{id}$, we obtain

$$\big(e(O_{id,i}, c_3)^{-1} \cdot c_{2,j^\star}\big)^{1/sk_{id}} = e(h_{u'_{id}}, h_{u'_{id}})^\gamma.$$

This is exactly the shared pairing value used to hide the masking value in the ciphertext.

Next, since

$$c_{4,j^\star} = H\Big(e(h_{u'_{id}}, h_{u'_{id}})^\gamma\Big) \oplus \beta,$$

the querier can recover the masking value as

$$\beta = c_{4,j^\star} \oplus H\Big[\big(e(O_{id,i}, c_3)^{-1} \cdot c_{2,j^\star}\big)^{1/sk_{id}}\Big].$$

Finally, from

$$c_5 = H(e(g,g)^\beta) \oplus (pk_h \| sk_{h,u}),$$

the querier can recover the valid query key material:

$$pk_h \| sk_{h,u} = c_5 \oplus H(e(g,g)^\beta).$$

Therefore, any querier that is both registered and authorized under the access policy can correctly recover the query key material. This establishes the correctness of the authorized query-key recovery process in DeCart.

c) Correctness of AI Query.: After recovering the valid query key material, the querier encrypts the AI model under pk_h, while the data owner encrypts the outsourced data records under the same public key. By the correctness of threshold homomorphic encryption, the cloud server can homomorphically evaluate the encrypted AI model over the encrypted data records, and the resulting ciphertexts correspond exactly to the inference results in the plaintext domain. Then, using the server-side and user-side decryption shares, the authorized data querier can correctly recover the final plaintext query result. Therefore, as long as the recovered query key material is valid, the encrypted AI query process in DeCart returns the correct query result.

d) Correctness of DeCart.:* The main difference in DeCart* lies in the organization of the public parameters and auxiliary parameters, which is used to reduce the storage overhead and improve scalability. Although its parameter organization is different from that of DeCart, the functional logic of registration verification, authorization checking, shared-value recovery, and the following threshold homomorphic AI query remains equivalent to that of DeCart.

Therefore, the above correctness analysis also applies to DeCart*. That is, in DeCart*, as long as a data querier is legally registered and satisfies the access policy, it can also correctly recover the query key material and obtain the correct AI query result.

E. Complexity Analysis

In this section, we analyze the theoretical complexity of DeCart and DeCart* from three aspects: computation, communication, and storage. Let N be the maximum number of users, n be the number of users in each block, and $B = \lceil N/n \rceil$ be the number of blocks. We further use n_p to denote the size of the access policy, n_m to denote the number of data records, and $|M|$ to denote the size of the AI model.

a) Complexity of DeCart: In the Setup phase, DeCart generates n parameters $\{h_i\}$ and $n(n-1)$ pairwise parameters $\{H_{i,j}\}$, resulting in $O(n^2)$ setup computation and public-parameter storage cost. In KeyGen and Register, each user generates and verifies a linear number of auxiliary parameters, yielding $O(n)$ cost. Encrypt costs $O(n_p + n_m)$ in computation and communication. Check requires $O(n)$ search in the auxiliary parameter set. Query overhead is dominated by homomorphic evaluation: $O(n_m \cdot \text{HE-Eval}(|M|))$. Decrypt costs $O(n_r)$ for n_r returned results. Revoke costs $O(n)$ for parameter regeneration, plus $O(n_p)$ if policy-related ciphertexts are refreshed.

b) Complexity of DeCart*: The main optimization of DeCart* lies in reorganizing public and auxiliary parameters, reducing both setup computation and public-parameter storage from $O(n^2)$ to $O(n)$. The asymptotic complexities of KeyGen, Register, Encrypt, Check, Query, Decrypt, and Revoke remain the same as those of DeCart, as the functional procedures are unchanged.

VI. Security Analysis

We analyze the security of Decart and Decart* under the semi-trusted threat model. In this model, all entities honestly

follow the protocol specification but may try to infer private information from their local views. We mainly consider IND-CPA security against unauthorized AI queries and privacy leakage from outsourced data and encrypted AI models.

Theorem 1 (IND-CPA Security of Decart)**.** *Assume that the decisional bilinear Diffie–Hellman (DBDH) assumption holds in the underlying bilinear group, the hash function H is modeled as a random oracle or a secure key-derivation function, and the threshold homomorphic encryption scheme is IND-CPA secure. Then,* Decart *achieves IND-CPA security against any probabilistic polynomial-time adversary in the semi-trusted model.*

Proof. We prove the theorem by a standard hybrid argument. In Decart, the data owner encrypts data records under a homomorphic encryption key pk_h, while the corresponding user-side secret share $sk_{h,u}$ is protected by the mask $H(e(g,g)^{\beta})$. The value β is further protected in each policy component by the pairing-derived mask $H(e(h_{u'}, h_{u'})^{\gamma})$. Only an authorized user whose identity satisfies the policy can use its secret key and auxiliary opening to derive the required pairing value and recover β. An unauthorized user, even with its own secret key and auxiliary parameters, cannot derive the correct pairing term for an identity not assigned to it.

Suppose there exists an adversary $\mathcal{A}$ that can distinguish the challenge ciphertexts in Decart with non-negligible advantage. We construct a simulator $\mathcal{B}$ that uses $\mathcal{A}$ to break either the DBDH assumption or the IND-CPA security of the underlying homomorphic encryption scheme. First, $\mathcal{B}$ embeds the DBDH challenge into the policy-related ciphertext components. If the DBDH challenge is real, then the ciphertext distribution is identical to that in the real Decart security game. If the DBDH challenge is random, then the masks used to protect β are indistinguishable from uniformly random strings. Therefore, β is hidden from any unauthorized adversary. Second, after replacing the mask for β with a random value, the value $H(e(g,g)^{\beta})$ also becomes indistinguishable from a random string. Thus, the encapsulated key material $(pk_h \| sk_{h,u})$ leaks no information to the adversary. The remaining challenge components consist of homomorphic ciphertexts of data records or AI model parameters. By the IND-CPA security of the threshold homomorphic encryption scheme, these ciphertexts are indistinguishable for any two equal-length challenge messages. Therefore, if $\mathcal{A}$ has non-negligible advantage in distinguishing the challenge ciphertexts of Decart, then $\mathcal{B}$ can break either the DBDH assumption or the IND-CPA security of the underlying homomorphic encryption scheme, which contradicts our assumptions. Hence, Decart is IND-CPA secure. □

Theorem 2 (IND-CPA Security of Decart*)**.** *Assume that the corresponding bilinear exponent assumption, e.g., the n-BDHE assumption, holds in the underlying bilinear group, the hash function H is modeled as a random oracle or a secure key-derivation function, and the threshold homomorphic encryption scheme is IND-CPA secure. Then,* Decart* *achieves IND-CPA security against any probabilistic polynomial-time adversary in the semi-trusted model.*

Proof. The proof is similar to that of Decart, except that Decart* uses compact exponent-chain public parameters of the form $h_i = g^{z^i}$ to reduce users' stored parameters. In Decart*, an authorized user derives the policy-dependent pairing value $e(h_{u'}, h_{n+1-u'})^{\gamma}$ using its secret key and auxiliary opening. This value is then used to recover β, which further protects $(pk_h \| sk_{h,u})$. For an unauthorized adversary, the required pairing value is hidden under the underlying bilinear exponent assumption. More specifically, without the correct account-level secret key and auxiliary opening, the adversary cannot compute $e(h_{u'}, h_{n+1-u'})^{\gamma}$ for an authorized identity. If the adversary can distinguish the resulting mask from a random value, then we can build a simulator that solves the underlying n-BDHE-type challenge. After replacing this policy-derived mask with a uniformly random string, β becomes information-theoretically hidden from unauthorized entities. Consequently, the mask $H(e(g,g)^{\beta})$ also hides $(pk_h \| sk_{h,u})$. The adversary's remaining view only contains homomorphic ciphertexts of data records, encrypted AI models, and encrypted query results. These ciphertexts are indistinguishable under the IND-CPA security of the threshold homomorphic encryption scheme. Therefore, any non-negligible advantage against Decart* implies a non-negligible advantage against either the underlying bilinear exponent assumption or the IND-CPA security of the homomorphic encryption scheme. This contradicts the assumptions. Hence, Decart* is IND-CPA secure. □

Theorem 3 (Resistance to Collusion Attacks)**.** *Assume that the hardness assumptions used in Theorems above hold and that the threshold homomorphic encryption scheme is secure against fewer-than-threshold collusions. Then,* Decart *and* Decart* *resist collusion attacks among semi-trusted entities, including unauthorized data queriers, data owners, the key curator, and cloud servers, as long as no authorized user deliberately releases its recovered key material or plaintext query results.*

Proof. We consider several possible collusion cases. First, consider collusion among unauthorized data queriers. Each querier only holds its own secret key and auxiliary parameters. Since the recovery of β is bound to a specific account-level identity in the access policy, auxiliary parameters from different unauthorized users cannot be algebraically combined to produce the missing authorized opening. Therefore, a group of unauthorized users cannot recover β or $(pk_h \| sk_{h,u})$ unless at least one of them is explicitly authorized by the policy. Second, consider collusion between a cloud server and unauthorized data queriers. The cloud server stores policy-related ciphertexts and a server-side threshold decryption share $sk_{h,s}$. However, the cloud server does not know the user-side share $sk_{h,u}$ unless it is recovered by an authorized user. Since unauthorized queriers cannot recover $(pk_h \| sk_{h,u})$, collusion with the cloud server cannot complete the threshold decryption process or execute valid unauthorized AI queries. Third, consider the key curator. The key curator observes public keys, personal auxiliary parameters, and auxiliary states. These values are group elements generated from users' secret exponents and are used only for registration and authorization checking. Under the corresponding bilinear hardness assumptions, they do not

reveal the protected value β, the homomorphic secret key share $sk_{h,u}$, or plaintext data/model information. Therefore, the key curator cannot independently decrypt outsourced data records or AI query results. Fourth, consider collusion among cloud servers in the multi-cloud extension. The homomorphic secret key is split into multiple server-side shares and one user-side share. Fewer-than-threshold cloud servers cannot reconstruct the server-side decryption capability, and even above-threshold partial decryption still requires the authorized user's share to obtain the final plaintext result. Therefore, the multi-cloud extension mitigates collusion between a single cloud server and unauthorized queriers. Hence, under the stated assumptions, colluding semi-trusted entities cannot bypass the account-level query policy or learn encrypted data records and AI models beyond their authorized outputs. □

VII. Performance Evaluations

A. Settings of Experiments

Setup.

All experiments are conducted on the same machine under a unified software environment to ensure fairness and reproducibility. The experimental platform is equipped with an Intel Core Ultra 9 285H processor, 32 GB RAM, and runs 64-bit Windows 11. Our implementation is based on Python 3.11.9 and uses bn256, blspy, cryptography, phe, and TenSEAL for cryptographic operations, while numpy, torch, and torchvision are used for numerical computation and model processing. Unless otherwise specified, all compared schemes are evaluated under the same parameter settings, including the total number of users N, block size n, number of records, record dimension, and policy size.

Datasets and Models.

The experiments in this project mainly use synthetic data records and query workloads generated by the experiment scripts. For system-level overhead evaluation, including Setup, KeyGen, Register, communication cost, and storage cost, we vary the number of data records, record dimension and block size to analyze the performance of different schemes under different settings. For model-related evaluation, we use the query tasks implemented in this project. Since different schemes do not support exactly the same model types, some comparisons are conducted only within the range of models supported by the corresponding schemes. In particular, SecPQ only supports decision-tree models, so it is not included in comparisons on more general model types, such as neural networks, and we evaluate only the other schemes that support those models. In addition, to show the extra cost introduced by security mechanisms, we report plaintext results in some experiments as a non-secure reference, but we do not treat plaintext as a formal comparison baseline.

Baselines for Comparison.

We compare DeCart and DeCart* with several representative baseline schemes introduced earlier, including a naive CCS-2023-style baseline [24], [25], a server-aided baseline [22], an offline key-distribution baseline [23], and SecPQ [19]. These baselines reflect different system design choices for controlled encrypted AI queries. In addition, we report plaintext results as an efficiency upper bound without privacy protection.

For decision-tree tasks, we mainly compare DeCart, DeCart*, the naive CCS-2023-style scheme, the server-aided scheme, the offline scheme, and SecPQ, We also report plaintext results as a reference baseline, but plaintext is not included in the formal comparison. For dot-product and neural-network tasks, since SecPQ does not support these models, we compare only the remaining schemes. Beyond cross-scheme comparisons, we also study DeCart and DeCart* separately by examining their size under different parameter scales and their performance in the revoke phase, further highlighting their characteristics in structural optimization, space efficiency, and system maintenance.

B. Experimental Results

In this section, we will present the experimental results from two aspects: baseline comparison and model scalability. First, we compare DeCart and DeCart* with the baseline schemes introduced earlier to evaluate their performance in terms of computation cost and communication cost. Then, we further present the storage cost and revocation ability of DeCart and DeCart* to demonstrate the applicability and scalability of the DeCart family.

1) Baseline Comparison:

a) Computation Cost: Overall, the running time of all schemes increases significantly as the number of data records grows. In most settings, DeCart* performs better than or close to DeCart, which suggests that its structural optimization can bring practical benefits in the online query stage.

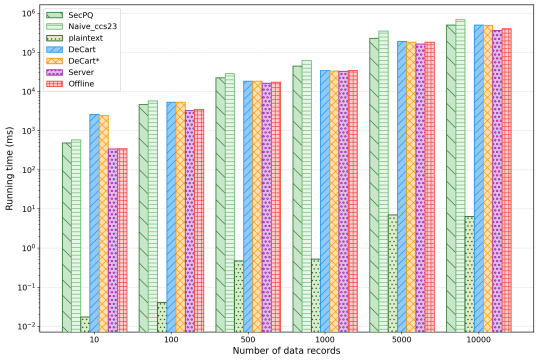
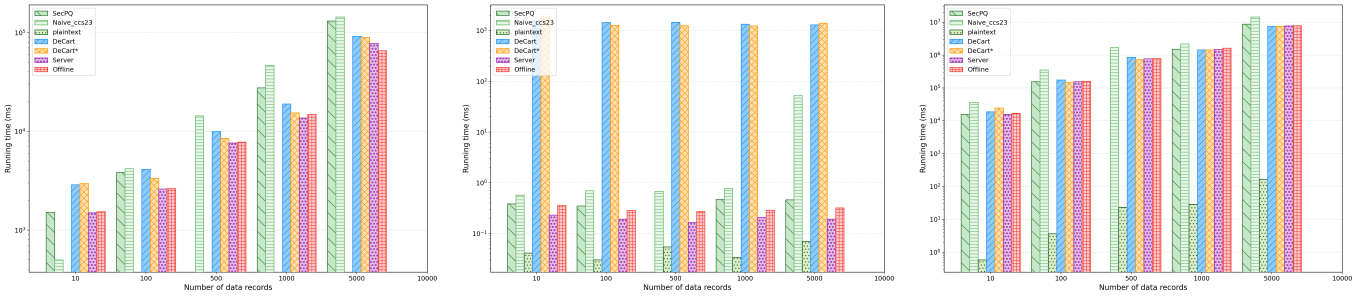
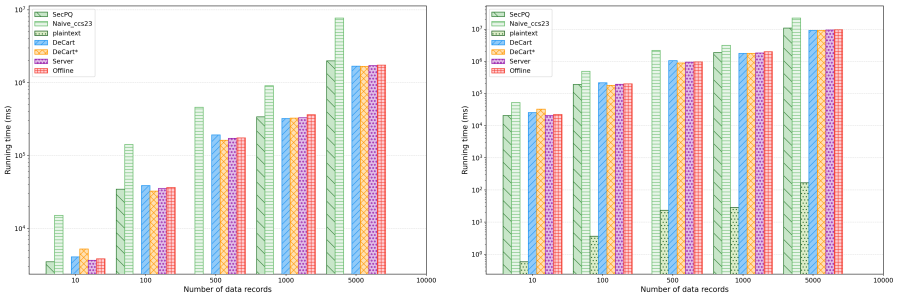


Fig. 4: Computation costs versus data size for the decision tree model with a single querier ($q = 1$).

For decision tree tasks with 1 query, as Fig.4, taking 10,000 records with 10,000 dimensions as an example, DeCart* further reduces the total running time by about 1.6% compared with DeCart. DeCart and Decart* achieve about 1.38x/1.40x and 1.01x/1.03 speedup over naive CCS-2023 and SecPQ. However, neither of them shows a clear advantage over the Server and Offline schemes. As Fig. 5, For decision tree tasks with 100 queries, taking 5,000 records with 5,000 dimensions as an example, DeCart and DeCart* achieve speedups of about 2.39x/2.41x, 1.17x/1.17x, 1.02x/1.03x, and 1.04x/1.05x over naive CCS-2023, SecPQ, Server and Offline. These results indicate that the DeCart family still has good computational advantages for large-scale decision tree tasks.



(a) Encrypt (b) Check (c) Query



(d) Decrypt (e) Total

Fig. 5: Computation costs versus data size for the decision tree model with multiple queriers ($q = 100$), including encrypt, check, query, decrypt, and total running time.

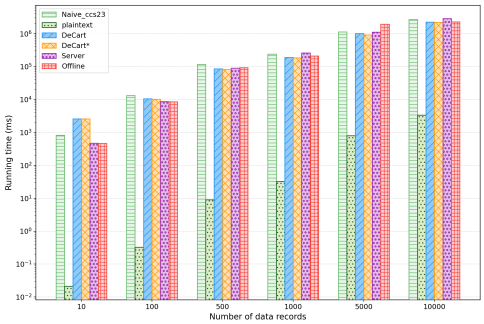


Fig. 6: Computation costs versus data size for the dot model with a single querier ($q = 1$).

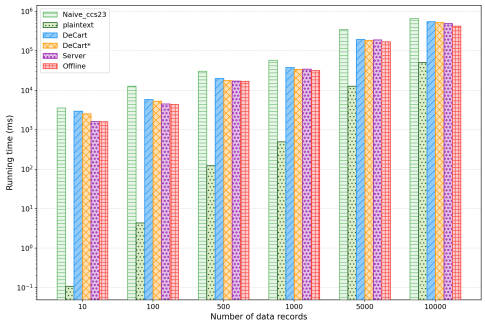


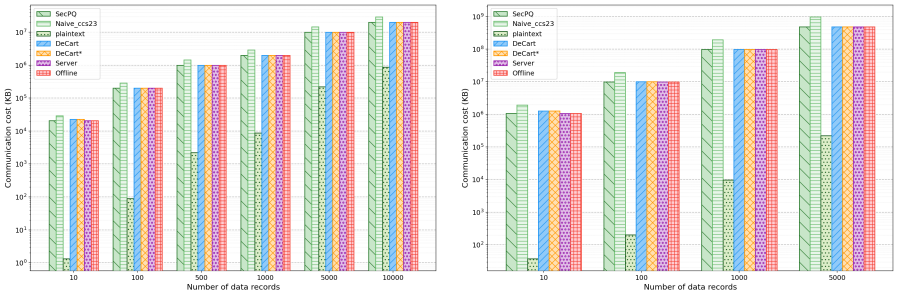
Fig. 7: Computation costs versus data size for the neural network model with a single querier ($q = 1$).

For the dot-product task, as Fig. 6, using 10,000 records with 10,000 dimensions, DeCart and Decart* achieve speedups of about 1.19x/1.21x, 1.28x/1.30x, and 1.02x/1.05x over naive CCS-2023, Server, and Offline, respectively. For the neural-network task under the same setting, as Fig. 7, DeCart and Decart* achieve about 1.18x/1.25x speedup over naive CCS-2023. Although DeCart is still slower than Server and Offline in this task, DeCart* further narrows the gap.

Totally, considering the results of decision tree, dot-product, and neural-network tasks at different comparable scales, the average speedups of DeCart and DeCart* over the other baselines are about 1.19x and 1.24x, respectively. This shows that the DeCart family can maintain strict access control while still achieving good overall computational efficiency, with DeCart* showing more robust overall performance.

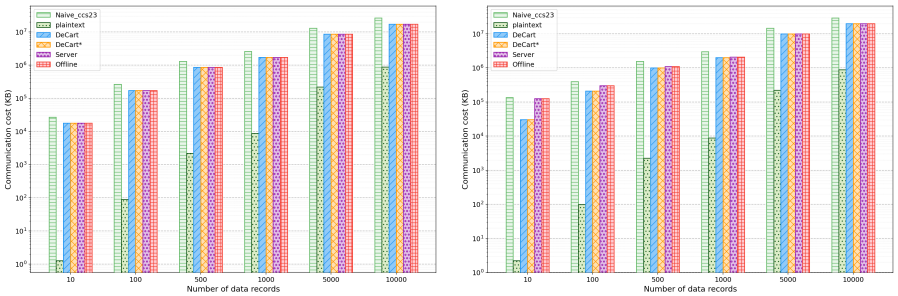
b) Communication Cost: In terms of communication cost, Fig. 8 shows the communication overhead of different schemes for decision tree tasks with 1 and 100 queries, as well as for dot-product and neural-network tasks. Overall, the communication cost of all schemes also increases significantly as the number of data records grows. Unlike the computation results, except for naive CCS-2023, the other methods have almost the same communication cost in most settings, and there is also little difference between DeCart and DeCart*. This suggests that the advantage of DeCart* mainly appears in the computation stage and does not cause extra communication cost. It also shows that DeCart and DeCart* can maintain strict access control while achieving communication efficiency comparable to the other schemes.

2) Storage Size and Parameter Scalability: From the overall cost caused by parameter scaling, as shown in Fig. 9, DeCart* shows better scalability in computation efficiency. The three performance figures for Setup, KeyGen, and Register show that, as the block size n increases, the running time of DeCart grows much faster than that of DeCart*. For example, when $n = 512$, the Setup, KeyGen, and Register times of DeCart are about 650 times, 7.9 times, and 12.0 times those of DeCart*, respectively. If we combine the time of these three



(a) Decision tree (q=1)

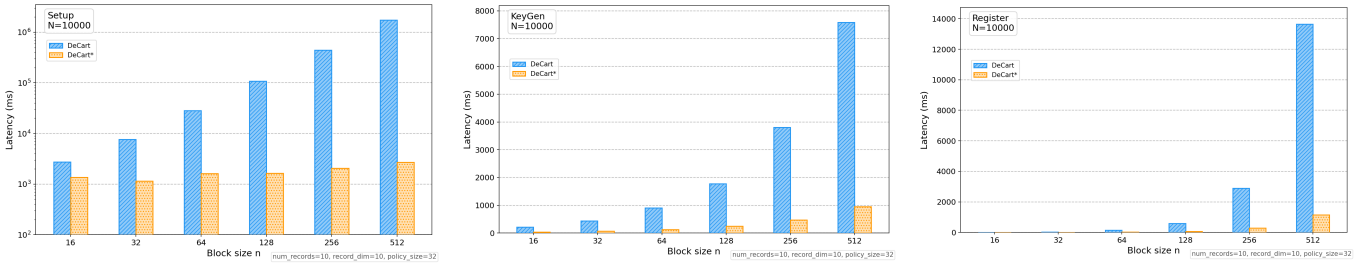
(b) Decision tree (q=100)



(c) Dot

(d) Neural network

Fig. 8: Communication costs versus data size for the different models with different numbers of queriers.



(a) Latency of Setup vs block size

(b) Latency of KeyGen vs block size

(c) Latency of Register vs block size

Fig. 9: Latency of setup, key generation and registration under different block sizes.

stages, the total time of DeCart is about 370 times that of DeCart*. This shows that DeCart* has a clear advantage in the overall system preparation and registration process, and it can control the computation cost caused by parameter scaling more effectively.

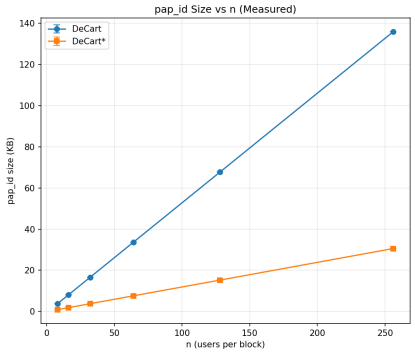
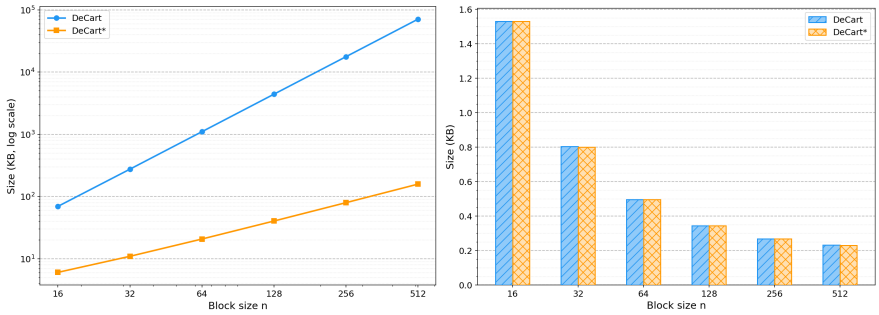


Fig. 10: PAP-id size versus block size.

Figure 10 shows how the size of a single-user `pap_id` changes with the block size n. The results show that the storage cost of both schemes increases as n becomes larger, but DeCart grows faster and stays clearly higher than DeCart* over the whole tested range. From the overall trend, the single-user `pap_id` storage cost of DeCart is about 4 to 5 times that of DeCart*. This indicates that the optimization of DeCart* in parameter organization not only reduces computation cost, but also improves space efficiency, giving it better scalability in large-scale scenarios.

In addition, for the storage cost, Fig.11 further shows that DeCart* also has better storage efficiency. As the block size n increases, the final Total Parameter Size of both schemes increases, but DeCart grows faster and is about 4.5 times larger than DeCart*. Separately, CRS and AUX show a similar increasing trend, while PP gradually decreases as n becomes larger, and the difference between the two schemes remains relatively small. Overall, DeCart* can reduce the storage pressure caused by public structures and auxiliary information more effectively, which gives it better space scalability in large-scale settings.

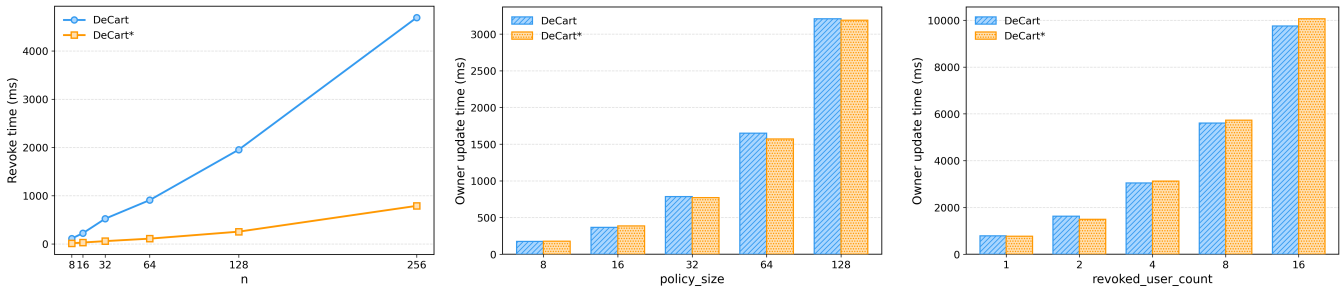
3) Revocation Evaluation: Figure 12 shows the time overhead of DeCart and DeCart* in the revocation phase, and examines the effects of block size n, policy size, and the number of revoked users. The results for revocation time



(a) CRS Size (b) PP Size

(c) Aux Size (d) Total Parameter Size

Fig. 11: Storage breakdown versus block size n.



(a) Revocation Overhead vs. Block Size (b) Revocation Overhead vs. Policy Size (c) Revocation Overhead vs. Number of Revoked Users

Fig. 12: Revocation overhead under different block sizes, policy sizes, and numbers of revoked users.

versus n show that, as the block size increases, DeCart* grows more slowly and remains clearly lower than DeCart, which indicates better scalability in revocation. Over the whole tested range, DeCart* keeps an average performance advantage of about 7 to 8 times in the revocation phase. In contrast, the owner update time increases as the policy size and the number of revoked users grow, but the difference between DeCart and DeCart* is not obvious in these two cases. Therefore, these results mainly reflect the effect of update scale on system maintenance cost. Overall, the main advantage of DeCart* in revocation lies in its lower revocation time when the block size is larger.

VIII. Conclusion

In this paper, we proposed two decentralized authorized AI query schemes for cloud-based IoT, DeCart and DeCart*. The proposed schemes protect the privacy of the data, the query, and the model, while supporting fine-grained access control, user registration, and user revocation, making them more suitable for dynamic query settings. Based on DeCart, DeCart* further reduces system overhead and improves overall efficiency.

Our theoretical analysis shows that the proposed schemes satisfy the required security and functionality goals. Experimental results further demonstrate their feasibility and effectiveness. Compared with DeCart, DeCart* achieves better overall performance in storage, computation, and communication costs, and shows more stable behavior at larger scales. At the same time, the results also show that the DeCart family is not limited to decision-tree queries, but can be extended to support more complex model queries, including dot-product and neural network models.

Overall, DeCart and DeCart* provide a complete and effective solution for privacy-preserving queries. Both schemes satisfy the desired security and functionality requirements while supporting dynamic user management and multi-model query extension, and DeCart* further offers better overall performance in storage, computation, and communication overhead.

REFERENCES

[1] K. Lam, X. Lu, L. Zhang, X. Wang, H. Wang, and S. Q. Goh, "Efficient fhe-based privacy-enhanced neural network for ai-as-a-service," *IACR Cryptol. ePrint Arch.*, vol. 2023, p. 647, 2023. [Online]. Available: https://eprint.iacr.org/2023/647

[2] A. Luqman, R. Mahesh, and A. Chattopadhyay, "Privacy and security implications of cloud-based ai services: A survey," *CoRR*, vol. abs/2402.00896, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2402.00896

[3] C. Meurisch and M. Mühlhäuser, "Data protection in ai services: A survey," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 40:1–40:38, 2022. [Online]. Available: https://doi.org/10.1145/3440754

[4] A. Liu, W. Jiang, S. Huang, and Z. Feng, "Multi-modal integrated sensing and communication in internet of things with large language models," *IEEE Internet Things Mag.*, vol. 8, no. 5, pp. 103–111, 2025. [Online]. Available: https://doi.org/10.1109/MIOT.2025.3575888

[5] B. Muthu, C. B. Sivaparthipan, G. Manogaran, R. Sundarasekar, S. N. Kadry, A. Shanthini, and A. A. Dasel, "Iot based wearable sensor for diseases prediction and symptom analysis in healthcare sector," *Peer-to-Peer Netw. Appl.*, vol. 13, no. 6, pp. 2123–2134, 2020. [Online]. Available: https://doi.org/10.1007/s12083-019-00823-2

[6] Y. Abudurexiti, G. Han, F. Zhang, and L. Liu, "An explainable unsupervised anomaly detection framework for industrial internet of things," *Comput. Secur.*, vol. 148, p. 104130, 2025. [Online]. Available: https://doi.org/10.1016/j.cose.2024.104130

[7] V. V. Zhebka, P. Skladannyi, S. Zhebka, S. Shlianchak, and A. Bondarchuk, "Methodology for predicting failures in a smart home based on machine learning methods," in *Proceedings of the Workshop Cybersecurity Providing in Information and Telecommunication Systems (CPITS 2024), Kyiv, Ukraine, February 28, 2024 (online)*, ser. CEUR Workshop Proceedings, V. Sokolov, V. Ustimenko, T. Radivilova, and M. Nazarkevych, Eds. CEUR-WS.org, 2024, pp. 322–332. [Online]. Available: https://ceur-ws.org/Vol-3654/paper27.pdf

[8] C. Chen, Z. Liu, S. Wan, J. Luan, and Q. Pei, "Traffic flow prediction based on deep learning in internet of vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3776–3789, 2021. [Online]. Available: https://doi.org/10.1109/TITS.2020.3025856

[9] A. Chaudhuri, S. Shukla, S. Bhattacharya, and D. Mukhopadhyay, "Secured and privacy-preserving gpu-based machine learning inference in trusted execution environment: A comprehensive survey," in *17th International Conference on COMmunication Systems and NETworks, COMSNETS 2025, Bengaluru, India, January 6-10, 2025*. IEEE, 2025, pp. 207–216. [Online]. Available: https://doi.org/10.1109/COMSNETS63942.2025.10885734

[10] T. Chen, Y. Tan, C. Li, Z. Zhang, W. Meng, and Y. Li, "Securecomm: A secure data transfer framework for neural network inference on CPU-FPGA heterogeneous edge devices," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 14, no. 4, pp. 811–822, 2024. [Online]. Available: https://doi.org/10.1109/JETCAS.2024.3491169

[11] M. Degeling, C. Utz, C. Lentzsch, H. Hosseini, F. Schaub, and T. Holz, "We value your privacy ... now take some cookies - measuring the gdpr's impact on web privacy," *Inform. Spektrum*, vol. 42, no. 5, pp. 345–346, 2019. [Online]. Available: https://doi.org/10.1007/s00287-019-01201-1

[12] Z. Zhang and Y. Zha, "Systematic construction of lawfulness of processing employees' personal information under china's personal information protection law," *Comput. Law Secur. Rev.*, vol. 50, p. 105853, 2023. [Online]. Available: https://doi.org/10.1016/j.clsr.2023.105853

[13] A. Thomaidou and K. Limniotis, "Navigating through human rights in ai: Exploring the interplay between gdpr and fundamental rights impact assessment," *J. Cybersecur. Priv.*, vol. 5, no. 1, p. 7, 2025. [Online]. Available: https://doi.org/10.3390/jcp5010007

[14] H. Sun, Y. Tan, L. Zhu, Q. Zhang, Y. Li, and S. Wu, "A fine-grained and traceable multidomain secure data-sharing model for intelligent terminals in edge-cloud collaboration scenarios," *Int. J. Intell. Syst.*, vol. 37, no. 3, pp. 2543–2566, 2022. [Online]. Available: https://doi.org/10.1002/int.22784

[15] J. Chen, M. Wang, Z. Cao, X. Dong, and L. Sun, "Secure and controllable cloud-edge collaborative data sharing scheme for wireless body area networks in iiot," *Comput. Secur.*, vol. 153, p. 104389, 2025. [Online]. Available: https://doi.org/10.1016/j.cose.2025.104389

[16] Y. Meng, B. Wang, Q. Xing, X. Wang, J. Liu, and X. Xu, "Bbad: Blockchain-based data assured deletion and access control system for iot," *Peer-to-Peer Netw. Appl.*, vol. 18, no. 2, p. 1, 2025. [Online]. Available: https://doi.org/10.1007/s12083-024-01881-x

[17] X. Qin, Y. Huang, Z. Yang, and X. Li, "Lbac: A lightweight blockchain-based access control scheme for the internet of things," *Inf. Sci.*, vol. 554, pp. 222–235, 2021. [Online]. Available: https://doi.org/10.1016/j.ins.2020.12.035

[18] W. Zhu, M. Wei, X. Li, and Q. Li, "Securebinn: 3-party secure computation for binarized neural network inference," in *Computer Security - ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part III*, ser. Lecture Notes in Computer Science, V. Atluri, R. D. Pietro, C. D. Jensen, and W. Meng, Eds. Springer, 2022, pp. 275–294. [Online]. Available: https://doi.org/10.1007/978-3-031-17143-7_14

[19] J. Liang, S. Guo, Z. Hong, E. Zhou, C. Zhang, and B. Xiao, "Secpq: Secure prediction queries on encrypted outsourced databases," *IEEE Trans. Dependable Secur. Comput.*, vol. 22, no. 5, pp. 4534–4548, 2025. [Online]. Available: https://doi.org/10.1109/TDSC.2025.3549052

[20] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. E. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19–24, 2016*, ser. JMLR Workshop and Conference Proceedings, M. Balcan and K. Q. Weinberger, Eds. JMLR.org, 2016, pp. 201–210. [Online]. Available: http://proceedings.mlr.press/v48/gilad-bachrach16.html

[21] C. Juvekar, V. Vaikuntanathan, and A. P. Chandrakasan, "Gazelle: A low latency framework for secure neural network inference," in *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15–17, 2018*, W. Enck and A. P. Felt, Eds. USENIX Association, 2018, pp. 1651–1669. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar

[22] L. Ferretti, M. Colajanni, and M. Marchetti, "Access control enforcement on query-aware encrypted cloud databases," in *IEEE 5th International Conference on Cloud Computing Technology and Science, CloudCom 2013, Bristol, United Kingdom, December 2–5, 2013, Volume 2*. IEEE Computer Society, 2013, p. 219. [Online]. Available: https://doi.org/10.1109/CloudCom.2013.172

[23] L. Zhou, Y. Zhu, and A. Castiglione, "Efficient k-nn query over encrypted data in cloud with limited key-disclosure and offline data owner," *Comput. Secur.*, vol. 69, pp. 84–96, 2017. [Online]. Available: https://doi.org/10.1016/j.cose.2016.11.013

[24] N. Glaeser, D. Kolonelos, G. Malavolta, and A. Rahimi, "Efficient registration-based encryption," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26–30, 2023*, W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds. ACM, 2023, pp. 1065–1079. [Online]. Available: https://doi.org/10.1145/3576915.3616596

[25] S. Garg, M. Hajiabadi, M. Mahmoody, A. Rahimi, and S. Sekar, "Registration-based encryption from standard assumptions," in *Public-Key Cryptography - PKC 2019 - 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14–17, 2019, Proceedings, Part II*, ser. Lecture Notes in Computer Science, D. Lin and K. Sako, Eds. Springer, 2019, pp. 63–93. [Online]. Available: https://doi.org/10.1007/978-3-030-17259-6_3

[26] Q. Wang, W. Ma, and W. Wang, "B-LNN: inference-time linear model for secure neural network inference," *Inf. Sci.*, vol. 638, p. 118966, 2023. [Online]. Available: https://doi.org/10.1016/j.ins.2023.118966

[27] D. T. K. Nguyen, D. H. Duong, W. Susilo, and Y. Chow, "Hesun: Homomorphic encryption for secure unbounded neural network inference," in *Security and Privacy in Communication Networks - 19th EAI International Conference, SecureComm 2023, Hong Kong, China, October 19-21, 2023, Proceedings, Part I*, ser. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, H. Duan, M. Debbabi, X. de Carné de Carnavalet, X. Luo, X. Du, and

M. H. A. Au, Eds. Springer, 2023, pp. 413–438. [Online]. Available: https://doi.org/10.1007/978-3-031-64948-6_21

[28] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *29th USENIX Security Symposium, USENIX Security 2020, August 12–14, 2020*, S. Capkun and F. Roesner, Eds. USENIX Association, 2020, pp. 2505–2522. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/mishra

[29] D. Boneh and M. K. Franklin, "Identity-based encryption from the weil pairing," in *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19–23, 2001, Proceedings*, ser. Lecture Notes in Computer Science, J. Kilian, Ed. Springer, 2001, pp. 213–229. [Online]. Available: https://doi.org/10.1007/3-540-44647-8_13

[30] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *Advances in Cryptology - ASIACRYPT 2001, 7th International Conference on the Theory and Application of Cryptology and Information Security, Gold Coast, Australia, December 9–13, 2001, Proceedings*, ser. Lecture Notes in Computer Science, C. Boyd, Ed. Springer, 2001, pp. 514–532. [Online]. Available: https://doi.org/10.1007/3-540-45682-1_30

[31] J. H. Cheon, A. Kim, M. Kim, and Y. S. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3–7, 2017, Proceedings, Part I*, ser. Lecture Notes in Computer Science, T. Takagi and T. Peyrin, Eds. Springer, 2017, pp. 409–437. [Online]. Available: https://doi.org/10.1007/978-3-319-70694-8_15

[32] D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. R. Rasmussen, and A. Sahai, "Threshold cryptosystems from threshold fully homomorphic encryption," in *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I*, ser. Lecture Notes in Computer Science, H. Shacham and A. Boldyreva, Eds. Springer, 2018, pp. 565–596. [Online]. Available: https://doi.org/10.1007/978-3-319-96884-1_19